

Il presente documento fornisce una descrizione tecnica esaustiva e approfondita delle funzionalità logiche e dei relativi flussi di controllo che caratterizzano la piattaforma **EthniGo**. L'analisi si basa sulla struttura del backend sviluppato in PHP (con estensione PDO per l'interfacciamento al database relazionale MySQL) e sul comportamento dell'interfaccia utente lato client.

1. Flusso di Registrazione Nuovi Utenti

Il processo di creazione di un nuovo account è gestito dal modulo descritto in `registra.php`. Il sistema applica una logica di onboarding controllata, delegando la scelta della password definitiva a una fase successiva al primo accesso.

Dettaglio del Meccanismo:

Step 1	Raccolta Dati: Il client compila un form HTML inviato tramite metodo POST contenente i campi anagrafici essenziali: Nome, Cognome, Username ed Email.
Step 2	Assegnazione Credenziali Temporanee: Il backend intercetta la richiesta ed assegna programmaticamente una password di default standardizzata, nello specifico la stringa <code>ABC123</code> .
Step 3	Cifratura e Persistenza: Prima della scrittura sul database, la password di default viene cifrata nativamente all'interno della query SQL sfruttando la funzione di hashing <code>SHA()</code> (equivalente a SHA-1). I dati vengono inseriti nella tabella <code>Utenti</code> garantendo l'univocità del campo Email grazie a un vincolo di <code>UNIQUE KEY</code> nel DBMS.

2. Sistema di Autenticazione (Login)

La procedura di verifica delle credenziali è implementata all'interno del file `login.php` e si occupa di verificare l'identità dell'utente, avviando formalmente la sessione di lavoro o forzando misure di sicurezza obbligatorie.

Dettaglio del Meccanismo:

- Ricezione e Hashing:** I parametri inviati tramite il modulo di login (Email e Password immessa in chiaro) vengono elaborati lato server. La password viene immediatamente convertita in un hash SHA-1 tramite la funzione PHP `sha1()`.

- **Interrogazione in Sicurezza (PDO):** Viene preparata una query SQL con parametri nominativi (`:email`) per estrarre la stringa di controllo memorizzata nel database, prevenendo attacchi di tipo SQL Injection.
- **Controllo Password di Default:** Se la corrispondenza degli hash è verificata, l'indirizzo email viene registrato nella variabile globale di sessione `$_SESSION['logged_user']` . Immediatamente dopo, il backend effettua un controllo specifico sulla stringa dell'hash ottenuto. Se l'hash corrisponde esattamente a `61768db8d1a38f1c16d3e6eea812ef423c739068` (ovvero il valore SHA-1 di `ABC123`), il sistema riconosce l'utilizzo della password temporanea.
- **Iniezione dello Script di Reindirizzamento:** Qualora l'utente stia usando la credenziale di default, viene generato un output JavaScript contenente un alert informativo (`< style="font-style: italic;">"Cambia la password di default con una a tua scelta!"`) e viene imposto un reindirizzamento forzato alla pagina di configurazione `psw.php` .

3. Logica di Cambio Password

La modifica delle credenziali (`psw.php` e `cambiaPswd.php`) agisce sia come sistema di aggiornamento volontario sia come sblocco di sicurezza per i neo-registrati.

Dettaglio del Meccanismo:

L'interfaccia client offre due campi di testo di tipo password con una comoda opzione nativa attivabile tramite checkbox (`togglePasswords()`) per commutare dinamicamente l'attributo HTML dei campi da *password* a *text*, migliorando l'esperienza utente in fase di digitazione.

Al momento del submit, l'elaborazione lato server segue una rigorosa logica a doppia verifica:

1. L'identità dell'utente viene letta in modo sicuro dalla sessione corrente (`$_SESSION['logged_user']`).
2. Il backend esegue una query di selezione per verificare che il valore hash della **vecchia password** inserita corrisponda millimetricamente a quanto conservato nel database.
3. Solo in caso di riscontro positivo, viene eseguita una seconda query di tipo UPDATE sulla tabella `Utenti`. La nuova stringa inserita viene processata direttamente nel database con la funzione `SHA(:pwd_nuova)` , aggiornando definitivamente la credenziale. Un messaggio di successo notifica l'utente e lo rimanda alla schermata di login per effettuare una nuova autenticazione pulita.

4. Funzionamento della Ricerca delle Città

La ricerca geografica rappresenta il primo livello del core funzionale di navigazione all'interno di `utente.php`. È implementata in modo asincrono per disaccoppiare la digitazione testuale dell'utente dalla memorizzazione dei dati di posizionamento precisi.

Dettaglio del Meccanismo:

- **Interfaccia di Input:** L'utente dispone di un campo di testo in cui digitare liberamente la città d'interesse (ad esempio "Milano" o "Roma"). Il sistema intercetta la pressione del tasto "Invio" tramite un listener JavaScript dedicato per prevenire il submit anticipato del form complessivo.
- **Integrazione API (TomTom):** La ricerca viene avviata mediante il pulsante "Cerca", che invoca la funzione JavaScript denominata `cityAPI()`. Questa routine interroga l'API esterna di geolocalizzazione (TomTom Geocoding API), autenticandosi tramite la chiave segreta configurata centralmente nel file isolato delle credenziali di ambiente.
- **Risoluzione dei Dati e Campo Hidden:** L'API restituisce un payload strutturato contenente le coordinate geografiche, il nome formattato e l'identificativo univoco della località. JavaScript elabora la risposta e popola dinamicamente un elemento di input nascosto nel form denominato `citta-scelta` (con id `#hidden-city`), bloccando il dato geografico definitivo pronto per essere passato al backend.

5. Funzionamento della Ricerca dei Ristoranti

Una volta che la città è stata validata ed agganciata al campo nascosto, l'utente può procedere all'interrogazione mirata dei punti di interesse gastronomici.

Dettaglio del Meccanismo:

La ricerca dei ristoranti combina i dati di posizionamento geografico precedentemente ottenuti con le categorie di cucina memorizzate nel sistema. Il flusso si articola come segue:

Generazione Dinamica del Filtro Cucine: All'interno del form, il menu a tendina (elemento HTML `<select name="opzioni">`) viene interamente generato lato server tramite PHP ed una connessione PDO che interroga la tabella `Categorie` del database.

Vengono ciclizzate tutte le tuple disponibili per stampare i tag d'opzione, impostando come valore visibile la descrizione testuale (es: *Africano*, *Greco*, *Messicano*, *Vietnamita*) e come attributo `value` l'ID numerico o alfanumerico della categoria (corrispondente ai codici standard dei Point of Interest delle mappe).

Inviando il form, lo script di destinazione (`trova-risto.php`) riceve via POST sia l'identificativo geografico della città sia l'ID della categoria culinaria. Utilizzando questi due parametri combinati, l'applicazione interroga i servizi di mappatura per estrarre l'elenco esatto dei ristoranti etnici che rispondono contemporaneamente a entrambi i criteri impostati.

6. Meccanismo di Gestione dei Preferiti

La persistenza e la personalizzazione dell'esperienza utente culminano nella possibilità di salvare i locali d'interesse, operazione orchestrata in modo trasparente dal file `salvaRistorante.pdf` (derivato dalla logica di `salvaRistorante.txt`).

Dettaglio del Meccanismo:

- 1. Comunicazione asincrona JSON:** Quando l'utente clicca sul comando di salvataggio di un ristorante dalla lista dei risultati, il client non ricarica l'intera pagina. Viene invece inviata una richiesta HTTP asincrona indirizzata al backend, configurando gli header per trasmettere un payload in formato JSON puro. Il server intercetta il flusso di input leggendolo direttamente tramite l'istruzione PHP `file_get_contents('php://input')` e decodificando l'oggetto in un array associativo tramite `json_decode()`.
- 2. Risoluzione delle Relazioni nel Database:** Il backend estrae l'identità dell'utente dalla sessione attiva (`$_SESSION['logged_user']`) e, tramite una query preparata sulla tabella `Utenti`, recupera l'ID numerico interno (chiave primaria) associato a tale email.
- 3. Mappatura dei Campi e Inserimento:** Il sistema mappa i dati estratti dal JSON ricevuto dall'API cartografica (gestendo opportunamente i valori mancanti tramite l'operatore di coalescenza nullo `??` per assegnare stringhe di fallback come "Nome non disponibile" o "Indirizzo non disponibile") e procede all'inserimento nella tabella relazionale `Preferiti`.

Schema Strutturale della Tabella Preferiti:

La tabella è strutturata per blindare l'integrità dei dati e supportare la cancellazione a cascata, come definito nel file di inizializzazione SQL del database:

Nome Campo	Tipo di Dato	Proprietà / Vincoli
ID_Utente	INT	Chiave Primaria Composta / Chiave Esterna associata a <code>Utenti(ID)</code> con istruzione <code>ON DELETE CASCADE</code> .
ID_Ristorante	VARCHAR(50)	Chiave Primaria Composta (ID univoco del locale fornito dall'API delle mappe).
Nome_Ristorante	VARCHAR(100)	NOT NULL (Denominazione ufficiale dell'esercizio commerciale).
Indirizzo	VARCHAR(255)	Memorizzazione della via e del numero civico formattato.
Telefono	VARCHAR(20)	Recapito telefonico del locale per contatto diretto.
SitoWeb	VARCHAR(255)	URL di riferimento del ristorante per consultazione menu.
DataAggiunta	TIMESTAMP	Valorizzato automaticamente al momento dell'inserimento col timestamp corrente (<code>CURRENT_TIMESTAMP</code>).

Grazie alla chiave primaria composta definita sulla coppia `(ID_Utente, ID_Ristorante)`, il database impedisce nativamente duplicazioni, garantendo che lo stesso utente non possa salvare più di una volta il medesimo ristorante, ottimizzando così le performance di indicizzazione e l'ordine dei dati.